

Tutorial for GORIC(A) evidence aggregation

Rebecca M. Kuiper

09 juli 2026

Contents

Examples with R code	2
Example 1: Relationship previous experience and trust	2
Example 2: Comparing surgical and non-surgical treatments for periodontal disease	5
Specifying sets of hypotheses in <code>evSyn</code>	18
Types of input for <code>evSyn</code>	20
using Estimates and Covariance Matrix	20
using GORIC(A) Values	21
using GORIC(A) Weights	22
using Log-likelihood and Penalty Values	23

This is a tutorial for performing GORIC(A) evidence aggregation using the `evSyn` function from the `restriktor` package. With GORIC(A) evidence aggregation, one aggregates / combines / synthesizes evidence / support for one or more central theories obtained from multiple (diverse / heterogeneous) studies using the GORIC(A).

Two examples using real data are provided. The first example is a step-by-step guide that illustrates GORIC(A) evidence aggregation for a central theory regarding a single relationship, using multiple heterogeneous/diverse studies. In the second example, GORIC(A) evidence aggregation is applied to several sets of multiple hypotheses that reflect central theories. Here, data from the research by Berkey and colleagues (1998) is used, which is available in the `metafor` library.

In both examples, we will show the input for and output of the `evSyn` function, together with the corresponding `evSyn` plot. The second example will also show `evSyn` plot options. In both examples, the function `leave1studyout` will be demonstrated as well. This function re-runs the evidence synthesis where each time omitting one study; it reports which hypothesis (or better: central theory) is preferred in each reduced set.

In each primary study, a central theory is reflected by a (statistical) hypothesis. This hypothesis may differ per primary study, as long as it reflects the central theory. Section 2 gives an overview of how to specify (study-specific) sets of (study-specific) hypotheses in the `evSyn` function.

The `evSyn` function currently accepts the following objects as input:

- a list of vectors with (standardized) parameter estimates;
- a list of vectors with log-likelihood values together with a list of vectors with penalty values;
- a list of vectors with GORIC(A) weights;
- a list of vectors with ratio of GORIC(A) weights;
- a list of vectors with GORIC(A) values;
- a list of objects of class `goric`;
- a `data.frame` of class `escalc` from the `metafor` package or a regular `data.frame` using the same labeling. In the first and last option, `evSyn` will calculate the GORICA first using `goric`. Some of the above options will be demonstrated below (in the examples and in Section 3).

Three types of approaches for evidence aggregation ('type_ev') are possible:

- "equal",
- "added", and
- "average".

The latter should be used when doing a multiverse analysis and, thus, when the aim is to average results from multiple analysis on one data set (due to different operationalization choices). The other two can be used when aggregating results from multiple studies.

In case of an equal-evidence approach, evidence from $N = 5$ studies with $n = 100$ observations (with support for the central theory) is the same as obtaining evidence from 1 study with $n = 5 * 100 = 500$ observations (with support for the central theory) – as in a meta-analysis. In the added-evidence approach, the evidence aggregated from $N = 5$ studies with $n = 100$ observations (with support for the central theory) is stronger than if the data came from a single study with $n = 5 * 100 = 500$ observations (with support for the central theory).

For ease, this tutorial only shows code for the added approach, but one can easily adjust it to the equal approach by specifying "type_ev = 'equal'" in the `evSyn` function.

Notes:

- In the examples, the evidence from 4 and 5 primary studies are aggregated. It is of course also possible to perform GORIC(A) evidence aggregation on many more studies.
- For (more) information regarding interpreting the GORIC(A) output, see 'Guidelines_output_GORIC' (<https://github.com/rebeccakuiper/Tutorials>).

Examples with R code

Example 1: Relationship previous experience and trust

This example uses data from four studies that investigate whether buyer's trust is affected by previous experiences between a buyer and a seller. The studies investigated this relationship with different statistical models; namely, with linear, probit, and three-level logistic regressions models (which make the parameter estimates incomparable).

The central theory is that previous experience is positively related to trust. The (study-specific) hypothesis that reflect this central theory is $H_+ : \beta_{past} > 0$, in all four studies.

In each study, the GORICA weights denote the support for the hypothesis of interest and thus the study-specific support for the central theory. Using GORICA evidence aggregation, the combined/aggregated/synthesized support/evidence for the central theory over all four diverse studies is obtained. Next, the corresponding R code is given.

Data preparation

Load the data and inspect it.

```
dat <- read.table("data/Kuiper2012estimates.txt", header = TRUE, sep = "\t")
dat
```

	study	year	estimate	se
1	Batenburg et al.	2003	0.090	0.029
2	Buskens and Raub	2002	0.140	0.054
3	Buskens and Weesie	2000	1.090	0.093
4	Buskens et al.	2010	1.781	0.179

The data consists of four rows and four columns. Each row represents a study. The columns are: **study**: the authors of the study, **year**: the year the study was published, **estimate**: the estimate of β_{past} , **se**: the standard error of the estimate. Only the estimates and their standard errors (or better, their variances) are needed to perform GORICA evidence aggregation.

Next you find code to load the `evSyn` function from the `restriktor` package, which is needed to perform the analysis. You can find also code to show the documentation for the function in the help-tab.

```
## First, install the package, if you have not done this already:
if (!require("restriktor")) install.packages("restriktor")

## Then, load the packages:
library(restriktor) # for the goric function

# If you want to use restriktor from github: if (!require('devtools'))
# install.packages('devtools') library(devtools)
# install_github('LeonardV/restriktor') library(restriktor) # for goric
# function

# print docs in the help-tab to view arguments and explanations for the
# function ?evSyn
```

Data for `evSyn`

In this case, the `evSyn` function requires some pre-processing of the data: it needs the estimates and their variances as a list of named vectors and as a list of (unnamed) covariance matrices, respectively. Each element in the list must represent a single study.

```
# set estimates as named vectors and store in list.
estimates <- lapply(dat$estimate, setNames, "beta_past")

# store variances (i.e., the square of the standard errors) in list of
# matrices.
covmats <- lapply(dat$se^2, as.matrix)
```

Hypotheses

One also needs to specify the (study-specific) theory-based hypothesis to be evaluated, using the name/label of the parameters. The expectation is that positive experiences with the seller in the past will have a positive effect on the buyer's trust. In this example, the same theory-based hypothesis can be used for all studies:

```
Hpos <- "beta_past > 0"
```

This hypothesis is evaluated against its complement; which equals $'beta_past < 0'$, in this case. Notably, when the hypothesis and its complement obtain equal support, then this would indicate support for the boundary, that is, the equality-constrained hypothesis $'beta_past = 0'$.

GORICA Evidence Synthesis

Using the created lists and hypothesis, run `evSyn` and obtain the results for GORICA Evidence Synthesis:

```
evSyn_trust <- evSyn(estimates, VCOV = covmats,
                     hypotheses = list(Hpos = Hpos),
                     # type_ev = 'added', # default
                     comparison = 'complement')

evSyn_trust
```

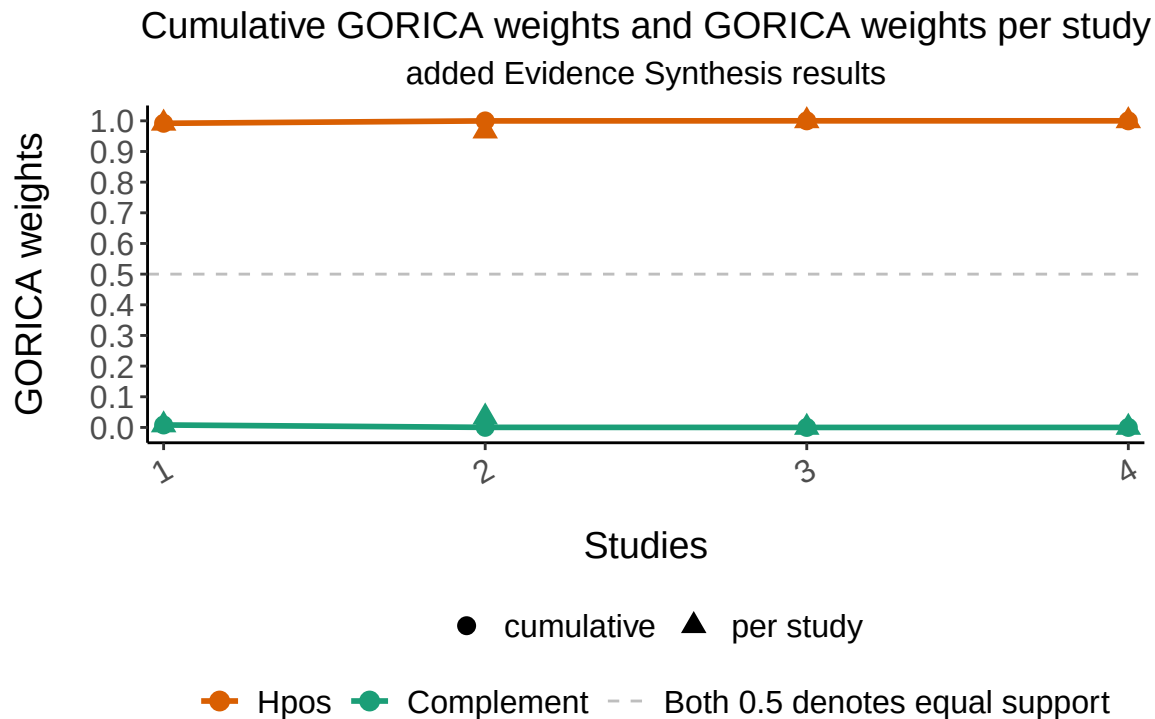
restriktor (0.6-50): added Evidence Synthesis results:

Input type 'est_vcov' detected: Parameter estimates and covariance matrix

```
Final GORICA weights:
      Hpos  Complement
      1.000      0.000
---
```

```
Ratio final GORICA weights:
      vs. Hpos  vs. Complement
Hpos      1.000      7.54e+54
Complement 0.000      1.000
```

```
#summary(evSyn_trust)
plot(evSyn_trust)
```



```
#round(evSyn_trust$Final_ratio_GORICA_weights, 2)
```

Here, one can see that the final/overall aggregated GORICA weights imply that H_+ (Hpos) has full support. Often a ratio of GORICA weights helps in interpreting the results. Such a ratio represents the relative aggregated support for one hypothesis (or better: central theory) over the other. It is presented in a $M \times M$ matrix, where M is the number of hypotheses in the set, where the diagonals represent the aggregated support of a hypothesis with itself, which is always one. To quantify the support for the central theory: The central theory H_+ has $7.5e+54$ (infinite) times more support than its complement, that is, all other possible theories. The conclusion, thus, would be that positive past experiences with a seller have a positive effect on the buyer's trust in that seller.

Sensitivity/Outlier analysis: Leave one study out

The function `leave1studyout` can be used to obtain insight into possible outlier or influential studies; or stated otherwise: to do a sensitivity analyses on the GORICA Evidence Synthesis. This function re-runs the evidence synthesis where each time omitting one study. It reports which hypothesis is preferred in each reduced set.

```
leave1out <- leave1studyout(evSyn_trust)
# leave1out
```

```
leave1out$OverallGoricaWeights
```

	Hpos	Complement
Leave Study 1 out:	1	1.638089e-53
Leave Study 2 out:	1	3.823582e-54
Leave Study 3 out:	1	8.955590e-26
Leave Study 4 out:	1	4.166967e-34

In this example, the conclusion will not change when one of the studies is left all. In all configurations, ‘Hpos’ is the overall preferred hypothesis (receiving full support).

Example 2: Comparing surgical and non-surgical treatments for periodontal disease

Background Information & Data

In the study by Berkey and colleagues (1998), surgical and non-surgical treatments for medium-severity periodontal disease are compared in five trials (i.e., 5 studies). Two outcomes reflect severity: attachment level (AL) and probing depth (PD) one year after the treatment. Higher attachment levels and lower probing depth are indicators of less severity, and are thus preferred.

The effect sizes to be aggregated are the (raw) mean differences, where non-surgical treatment is the reference category. The outcomes are calculated such that a positive value indicates that surgery was more effective in reducing severity than non-surgical treatment. In summary, a positive estimate indicates effectiveness of surgery in either increasing the attachment level or decreasing the probing depth.

First, install and call the `metafor` library to load data from Berkey (1998) and the `restriktor` library to load the `evSyn` function. If needed, it is possible to view the description of the data set and function with the `?` operator or the `help` command.

```
# To install the packages in R: if (!require('restriktor'))
# install.packages('restriktor')

# To install restriktor from github: if (!require('devtools'))
# install.packages('devtools') library(devtools)
# install_github('LeonardV/restriktor')
library(restriktor)
# print docs in the help-tab to view arguments and explanations for the
# function ?evSyn

# Data if (!require('metafor')) install.packages('metafor')
library(metafor)
dat <- dat.berkey1998
# print docs in the help-tab to view explanations for the data set
# ?dat.berkey1998
```

Let us inspect the `data.frame`.

```
View(dat)
```

Each `trial` is one study into the comparison of surgical and non-surgical treatments for severity of periodontal disease one year after treatment. Every study contains two rows in the `dat` data.frame, one for probing depth (PD) and one for attachment level (AL).

We can use the data.frame as input, but we could also use a list of vectors with (standardized) parameter estimates. For the latter, we need to prepare the data in such a way that it can be used as input for `evSyn`: store the estimates and covariance matrices in a list where each element of the list represents a single study.

```

# Store estimates in list of named vectors
estimates <- lapply(seq(1, nrow(dat), 2), function(i) setNames(c(dat$yi[i:(i + 1)]),
  c("PD", "AL")))
# Note that these names will be used in specifying the hypotheses.

# Store (co-)variances in list of (unnamed) variance-covariance matrices.
covmats <- lapply(seq(1, nrow(dat), 2), function(i) as.matrix(dat[i:(i + 1), c("v1i",
  "v2i")]))
# Note that it does not matter what the names/labels are; for the estimates it
# does matter.

```

Hypotheses

Below, you can find three possible (hypothetical) sets of central theories (and the corresponding hypotheses that reflect those).

Set 1

One may hypothesize from theory that surgery has a negative effect on attachment level, but a positive effect on probing depth. In each study, this can be reflected by: $H_{-+} : \theta_{AL} < 0; \theta_{PD} > 0$, where θ denotes the difference in means between surgical and non-surgical treatments.

There may also be reason to believe that surgery is too invasive, and thus non-surgery decreases severity on both outcomes compared to surgery: $H_{--} : \theta_{AL} < 0; \theta_{PD} < 0$.

Set 2

Alternatively, it can be the case that there is theory regarding the magnitude of the effect size and one wants to compare it to some cut-off value for that specific effect size type (e.g., when using Cohen's d or Hedges' g). In this example, there might be theory regarding the size of the mean differences stating: $H_{minAbsES} : |\theta_{AL}| > 0.2; |\theta_{PD}| > 0.2$. This states that the difference between the treatments on both outcomes is larger than 0.2.

Set 3

As a final example, it might be expected from theory that the absolute size of the effect for AL is smaller than for PD . In this example, the comparison of effect sizes can be represented by: $H_{absComp} : |\theta_{AL}| < |\theta_{PD}|$. This states that the absolute effect of surgery on attachment level is smaller than its absolute effect on probing depth.

Next, you can specify the (study-specific) hypotheses, where you now need the variables names or assigned names/labels of the estimates instead of the (Greek) parameter names:

```

# Set 1
H1.1 <- "AL < 0; PD > 0"
H1.2 <- "AL < 0; PD < 0"
# Set 2
H2 <- "abs(AL) > 0.2; abs(PD) > 0.2"
# Set 3
H3 <- "abs(AL) < abs(PD)"

```

GORICA Evidence Synthesis

Set 1 In the first set, H1.1 and H1.2 are investigated together, as if these are the hypotheses of interest (and each study uses the same specification, that is, the same set of hypotheses is applied to each study).

Next, two `evSyn` input specifications are shown. Both specification of course lead to same results, which are discussed subsequently.

```

# Study names #
# Instead of using numbers for the studies, one can also attach names to them.
# Here, the author names will be used, listed in the variable author in the data.frame:
study_names <- dat[dat$outcome=="AL",]$author

# GORICA evidence synthesis #
results_Set1_est <- evSyn(object = estimates, VCOV = covmats,
  hypotheses = list(H_neg_pos = H1.1, H_neg_neg = H1.2),
  #comparison = "unconstrained" # default
  #type_ev = "added" # default
  study_names = study_names
)

results_Set1_est

```

Input: list of vectors with parameter estimates

restriktor (0.6-50): added Evidence Synthesis results:

Input type 'est_vcov' detected: Parameter estimates and covariance matrix

Final GORICA weights:

	H_neg_pos	H_neg_neg	Unconstrained
---	0.997	0.000	0.003

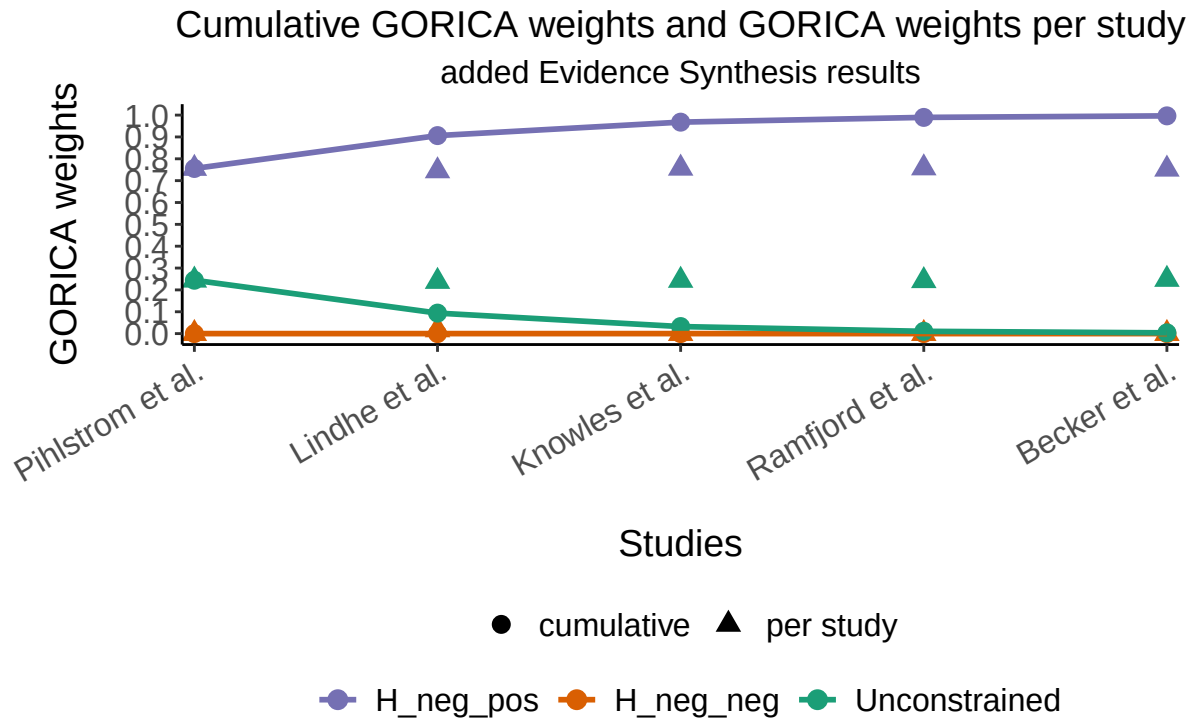
Ratio final GORICA weights:

	vs. H_neg_pos	vs. H_neg_neg	vs. Unconstrained
H_neg_pos	1.000	4.95e+34	285.368
H_neg_neg	0.000	1.000	0.000
Unconstrained	0.004	1.74e+32	1.000

```

# summary(results_Set1_est)
plot(results_Set1_est)

```



Input: data.frame If there would be only one (effect size) estimate, one can specify the data.frame as the input without an extra data-argument. When not specifying the data-argument 'outcome_col', the function assumes that the parameter of interest is called 'theta'; as it will also mention. The hypotheses should then thus also that labeling.

In this case, we have two parameters of interest and, thus, we have to specify the data-argument 'outcome_col'. This argument needs the name of the column that contains the outcome names. The hypotheses should then be specified using those outcome names. Here, the column is called 'outcome', which contains the names/labels 'AL' and 'PD' (which are also used in the hypothesis specification).

```
results_Set1 <- evSyn(object = dat,
  outcome_col = "outcome", # column with 'AL' & 'PB'
  hypotheses = list(H_neg_pos = H1.1, H_neg_neg = H1.2),
  #comparison = "unconstrained" # default
  #type_ev = "added" # default
  study_names = study_names
)
```

results_Set1

restriktor (0.6-50): added Evidence Synthesis results:

Input type 'escalc' detected: Parameter estimates and covariance matrix inherited from escalc object

Final GORICA weights:

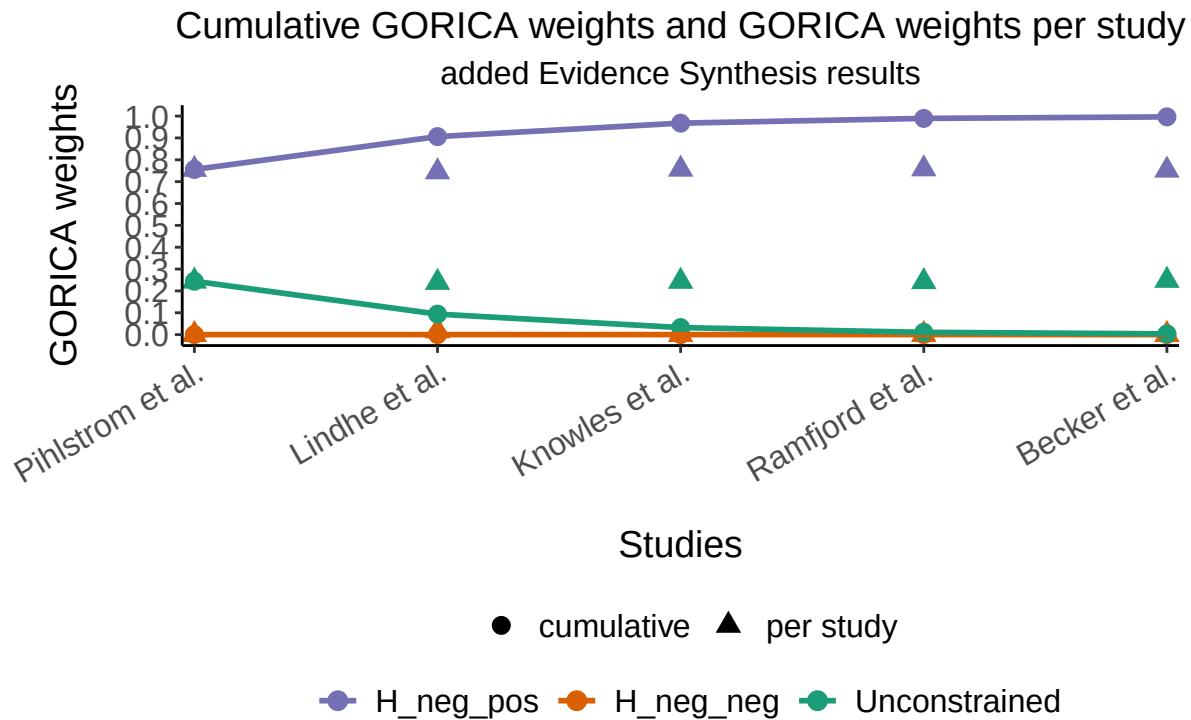
	H_neg_pos	H_neg_neg	Unconstrained
	0.997	0.000	0.003

Ratio final GORICA weights:

	vs. H_neg_pos	vs. H_neg_neg	vs. Unconstrained
H_neg_pos	1.000	4.95e+34	285.368

H_neg_neg	0.000	1.000	0.000
Unconstrained	0.004	1.74e+32	1.000

```
# summary(results_Set1)
plot(results_Set1)
```



Result Note that the hypotheses are passed as strings within the `list` function, where you can also denote hypothesis names. Note further that the complement cannot be used yet as a safeguard hypothesis; for now, this option is restricted to single order-restricted / theory-based / informative hypothesis. However, if the goal is to compare informative hypotheses, one could just as well use the unconstrained.

When inspecting the Final ratios of GORICA weights:

```
round(results_Set1$Final_ratio_GORICA_weights, 2)
```

	vs. H_neg_pos	vs. H_neg_neg	vs. Unconstrained
H_neg_pos	1	4.953573e+34	285.37
H_neg_neg	0	1.000000e+00	0.00
Unconstrained	0	1.735855e+32	1.00

one can see that H_{-+} ('H_neg_pos') is not weak, since the GORICA weights ratio with the unconstrained is larger than 1. Furthermore, that it is convincingly preferred over its alternative H_{--} ('H_neg_neg'): H_{-+} is practically an infinite amount of times more likely than H_{--} , given the data. Therefore, it can be concluded that surgical performance has a positive effect on probing depth, but a negative effect on attachment level.

Now, you may also want to investigate H_{-+} ('H_neg_pos') against its complement, which may be helpful for future research:

```
results_Set1_H1c <- evSyn(#estimates, VCOV = covmats,
  object = dat, outcome_col = "outcome",
  hypotheses = list(H_neg_pos = H1.1),
  #comparison = 'complement' # default
  #type_ev = "added" # default)
```

```
study_names = study_names
)
```

```
restriktor Message: The data contains multiple variables with a 'cluster_col' name, the following one is:
results_Set1_H1c
```

```
restriktor (0.6-50): added Evidence Synthesis results:
```

```
Input type 'escalc' detected: Parameter estimates and covariance matrix inherited from escalc object
```

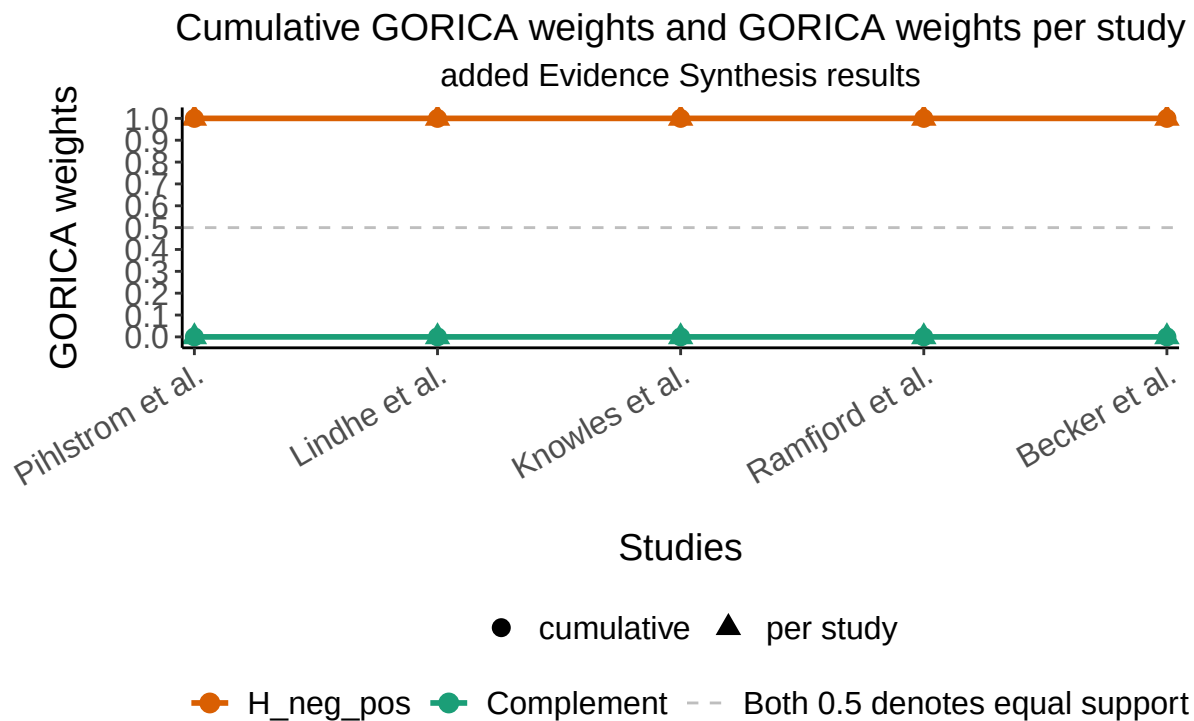
```
Final GORICA weights:
```

```
  H_neg_pos  Complement
      1.000      0.000
---
```

```
Ratio final GORICA weights:
```

```
          vs. H_neg_pos  vs. Complement
H_neg_pos          1.000          5.04e+218
Complement          0.000          1.000
```

```
# summary(results_Set1_H1c)
plot(results_Set1_H1c)
```



```
#round(results_Set1_H1c$Final_ratio_GORICA_weights, 2)
```

One can see that H_{-+} is also much more likely than its complement, that is, all other possible hypotheses / theories.

Set 2 In the second set, $H_{minAbsES}$ is the supposed central theory / hypothesis of interest, which will be denoted as H1 because that would be what you probably use in practice when evaluating this set. Notably, when not specifying a hypothesis label, H1 will also be, by default, the name of this hypothesis.

H1 states that the mean difference between surgery and non-surgery is larger than 0.2 for both outcomes. Note that the same hypothesis specification can be used for all studies.

```
results_Set2 <- evSyn(object = dat, outcome_col = "outcome",
                     hypotheses = list(H1 = H2),
                     #comparison = 'complement' # default
                     study_names = study_names
                     )
```

```
restriktor Message: The data contains multiple variables with a 'cluster_col' name, the following one is
results_Set2
```

```
restriktor (0.6-50): added Evidence Synthesis results:
```

```
Input type 'escalc' detected: Parameter estimates and covariance matrix inherited from escalc object
```

```
Final GORICA weights:
```

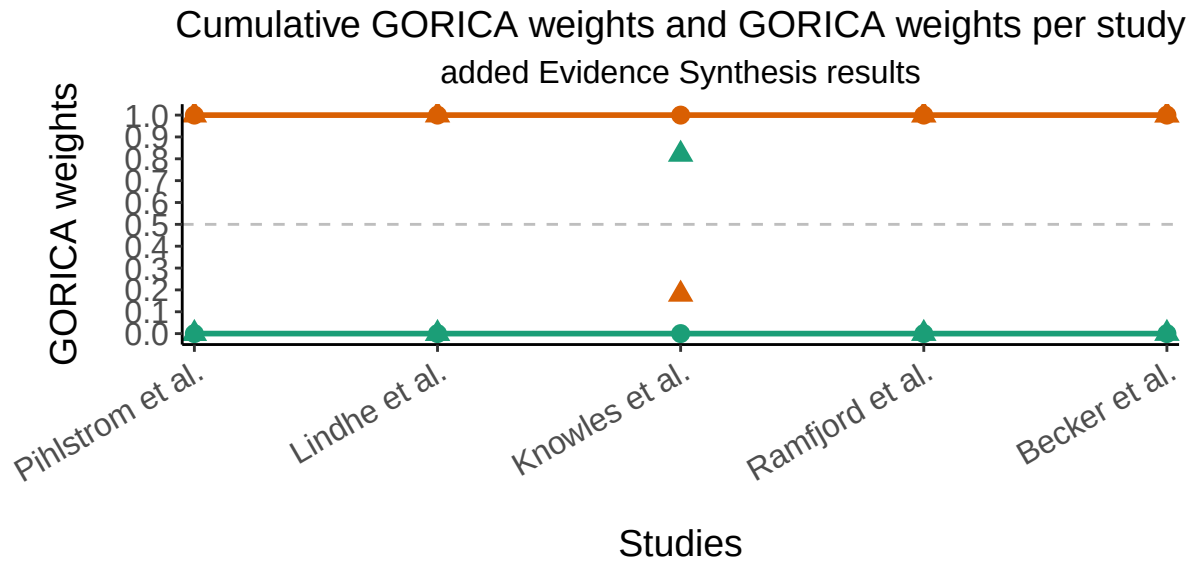
	H1	Complement
	1.000	0.000

```
---
```

```
Ratio final GORICA weights:
```

	vs. H1	vs. Complement
H1	1.000	3.23e+63
Complement	0.000	1.000

```
# summary(results_Set2)
plot(results_Set2)
```



● cumulative ▲ per study

● H1 ● Complement - - Both 0.5 denotes equal support

```
#round(results_Set2$Final_ratio_GORICA_weights, 2)
```

It can be concluded that H1 (i.e., $H_{minAbsES}$) is the best hypothesis (or better: central theory) as it is 3.2265319×10^{63} (> 1) times more likely than its complement. Hence, it is very likely that the absolute difference between the groups on both outcomes is larger than 0.2.

From the Study-specific results (available via the `summary` function) and the plot, it can be seen that the studies show somewhat mixed results: most preferring H1, while Study 3 clearly prefers the complement of H1. When aggregating the evidence, this results (in this case) in support for H1. Especially when there are many studies, it can be helpful to order the studies based on study-specific support for the overall best hypothesis (order_studies = “ascending” or “descending”). This way, it is more clear which studies do not find support for the overall best hypothesis, and one can look at study characteristics to find a possible alternative explanation. This of course does not affect the overall evidence/support.

```
results_Set2_asc <- evSyn(object = dat, outcome_col = "outcome",
  #hypotheses = list(H2), # default name will be H1
  hypotheses = list(H1 = H2), # this way you can label yourself
  order_studies = "ascending",
  study_names = study_names)
```

```
restriktor Message: The data contains multiple variables with a 'cluster_col' name, the following one is
results_Set2_asc
```

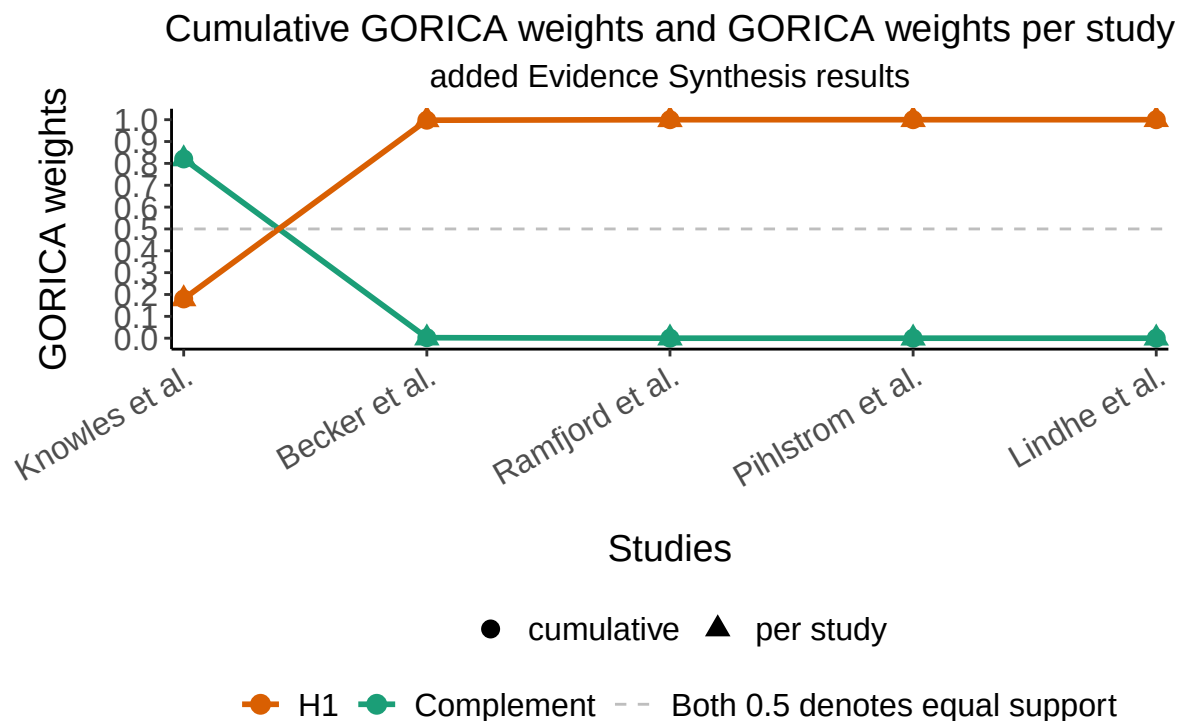
```
restriktor (0.6-50): added Evidence Synthesis results:
```

```
Input type 'escalc' detected: Parameter estimates and covariance matrix inherited from escalc object
```

```
Final GORICA weights:
      H1 Complement
1.000      0.000
---
```

```
Ratio final GORICA weights:
      vs. H1  vs. Complement
H1      1.000      3.23e+63
Complement 0.000      1.000
```

```
plot(results_Set2_asc)
```



If of interest, one can also specify their own order perhaps based on one of the study characteristics:

```
order_ascYear <- order(dat[dat$outcome == "AL", ]$year)
results_Set2_ascYear <- evSyn(object = dat, outcome_col = "outcome", hypotheses = list(H1 = H2),
  study_names = study_names, order_studies = order_ascYear)
```

```
restriktor Message: The data contains multiple variables with a 'cluster_col' name, the following one is
results_Set2_ascYear
```

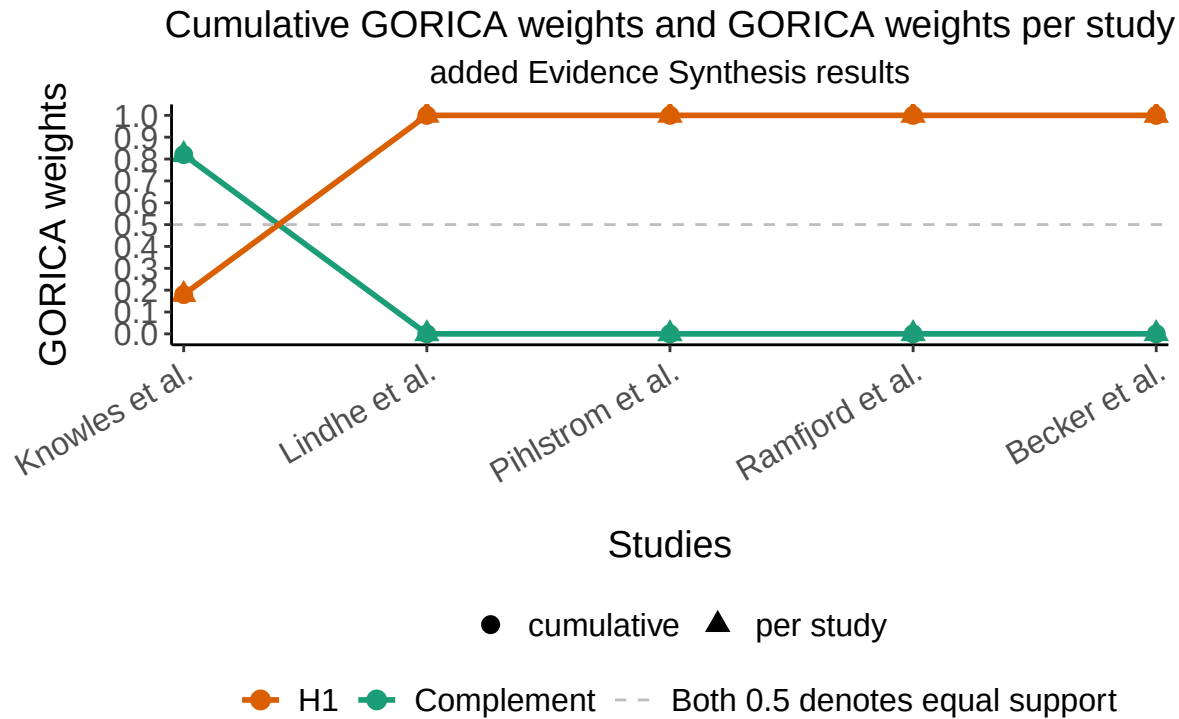
```
restriktor (0.6-50): added Evidence Synthesis results:
```

```
Input type 'escalc' detected: Parameter estimates and covariance matrix inherited from escalc object
```

```
Final GORICA weights:
      H1  Complement
      1.000      0.000
---
```

```
Ratio final GORICA weights:
      vs. H1  vs. Complement
H1      1.000      3.23e+63
Complement 0.000      1.000
```

```
plot(results_Set2_ascYear)
```



Sensitivity/Outlier analysis: Leave one study out The function `leave1studyout` can be used to obtain insight into possible outlier or influential studies; or stated otherwise: to do a sensitivity analyses on the GORICA Evidence Synthesis:

```
leave1out_Set2 <- leave1studyout(results_Set2)
# leave1out_Set2
leave1out_Set2$OverallGoricaWeights
```

	H1	Complement
Leave study nr. 1 called 'Pihlstrom et al.' out:	1	4.655076e-60
Leave study nr. 2 called 'Lindhe et al.' out:	1	4.354171e-11
Leave study nr. 3 called 'Knowles et al.' out:	1	6.792157e-65
Leave study nr. 4 called 'Ramfjord et al.' out:	1	1.107132e-60
Leave study nr. 5 called 'Becker et al.' out:	1	6.053645e-61

In this example, the results are not sensitive to the choice of set of studies: For all configurations, H1 (i.e., $H_{minAbsES}$) is the best hypothesis (or better: central theory), and it receives full support.

Set 3 In the third set, the hypothesis (or better: central theory) of interest is $H_{absComp}$, H1 in this set, which states that the negative effect of surgery on attachment level is smaller than the positive effect on probing depth. Also here, the same hypothesis specification can be used for all studies.

```
results_Set3 <- evSyn(object = dat, outcome_col = "outcome", hypotheses = list(H1 = H3),
  study_names = study_names)
```

restriktor Message: The data contains multiple variables with a 'cluster_col' name, the following one is

```
results_Set3
```

```
restriktor (0.6-50): added Evidence Synthesis results:
```

```
Input type 'escalc' detected: Parameter estimates and covariance matrix inherited from escalc object
```

```
Final GORICA weights:
```

H1	Complement
0.256	0.744

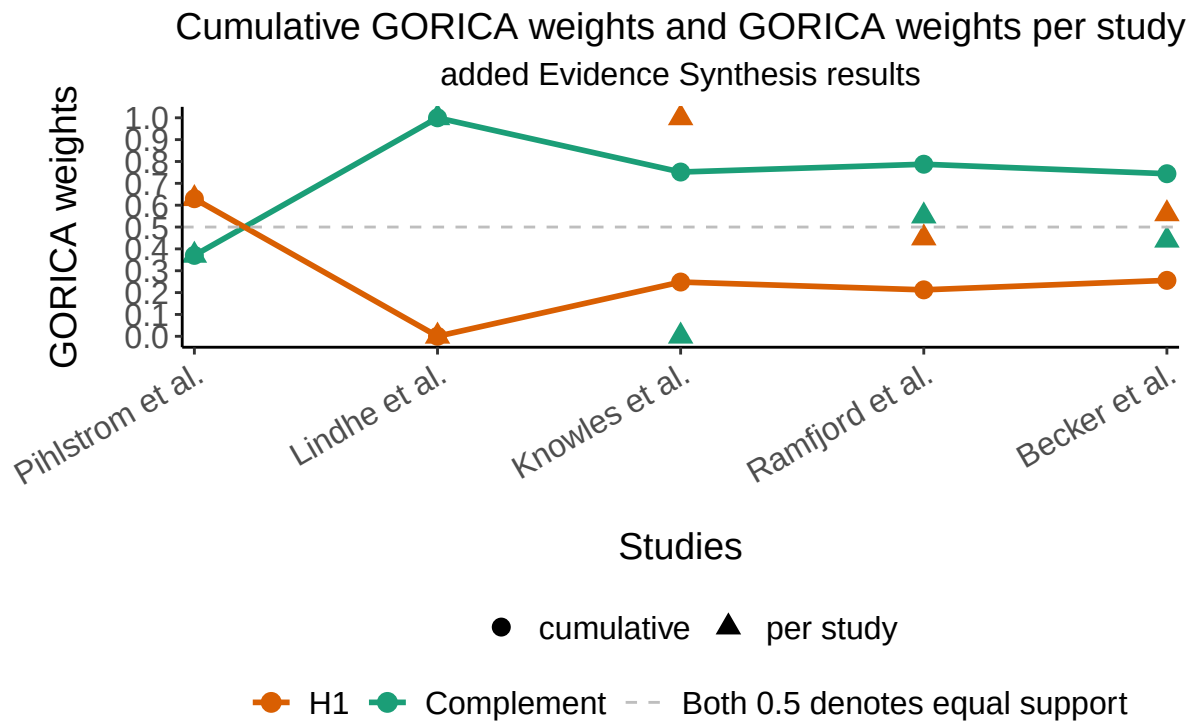
```
---
```

```
Ratio final GORICA weights:
```

	vs. H1	vs. Complement
H1	1.000	0.344
Complement	2.907	1.000

```
# summary(results_Set3)
```

```
plot(results_Set3)
```



```
# round(results_Set3$Final_ratio_GORICA_weights, 2)
```

The output shows that H1 (i.e., $H_{absComp}$) is 0.34 times more likely than its complement. Hence, the complement of H1 is 2.91 times more likely than H1 (i.e., $H_{absComp}$) itself.

From the Study-specific results (available via the `summary` function) and the plot, it can be seen that the studies show mixed results. When aggregating the evidence, this results (in this case) in support for the complement of H1.

Sensitivity/Outlier analysis: Leave one study out The function `leave1studyout` can be used to obtain insight into possible outlier or influential studies; or stated otherwise: to do a sensitivity analyses on the GORICA Evidence Synthesis:

```
leave1out_Set3 <- leave1studyout(results_Set3)
leave1out_Set3
```

```
$OverallGorica
```

		H1	Complement
Leave study nr. 1 called 'Pihlstrom et al.'	out:	0.5899321	-2.605503
Leave study nr. 2 called 'Lindhe et al.'	out:	-16.1222270	1.020767
Leave study nr. 3 called 'Knowles et al.'	out:	3.5805493	-14.553565
Leave study nr. 4 called 'Ramfjord et al.'	out:	2.8092848	1.078396
Leave study nr. 5 called 'Becker et al.'	out:	-1.5100157	-4.129029

```
$OverallGoricaWeights
```

		H1	Complement
Leave study nr. 1 called 'Pihlstrom et al.'	out:	0.1683008737	0.831699126
Leave study nr. 2 called 'Lindhe et al.'	out:	0.9998106070	0.000189393
Leave study nr. 3 called 'Knowles et al.'	out:	0.0001153924	0.999884608
Leave study nr. 4 called 'Ramfjord et al.'	out:	0.2962031578	0.703796842
Leave study nr. 5 called 'Becker et al.'	out:	0.2125693956	0.787430604

```
$OverallPrefHypo
```

Leave study nr. 1 called 'Pihlstrom et al.'	out:	"Complement"
Leave study nr. 2 called 'Lindhe et al.'	out:	"H1"
Leave study nr. 3 called 'Knowles et al.'	out:	"Complement"
Leave study nr. 4 called 'Ramfjord et al.'	out:	"Complement"
Leave study nr. 5 called 'Becker et al.'	out:	"Complement"

```
$type_ev
```

```
[1] "added"
```

```
$type
```

```
[1] "gorica"
```

```
attr("class")
```

```
[1] "leave1studyout.evSyn"
```

```
# leave1out_Set3$OverallGoricaWeights
```

In this example, the results are sensitive to the choice of set of studies. The conclusion even changes when study nr. 2 called 'Lindhe et al.' is left out. When one would leave that study out, H1 (i.e., $H_{absComp}$) is the overall best hypothesis (or better: central theory), receiving almost full support. When one of the other studies would be left out, then the overall GORICA weight for the complement varies between 0.704 and 1; where the overall support based on all studies was 0.744.

When ordering the studies based on study-specific support for the overall best hypothesis (central theory), it also becomes more clear that the second study is an influential study:

```
results_Set3_asc <- evSyn(object = dat, outcome_col = "outcome", hypotheses = list(H1 = H3),
  order_studies = "ascending", study_names = study_names)
```

```
restriktor Message: The data contains multiple variables with a 'cluster_col' name, the following one is
```

```
results_Set3_asc
```

```
restriktor (0.6-50): added Evidence Synthesis results:
```


Input type 'escalc' detected: Parameter estimates and covariance matrix inherited from escalc object

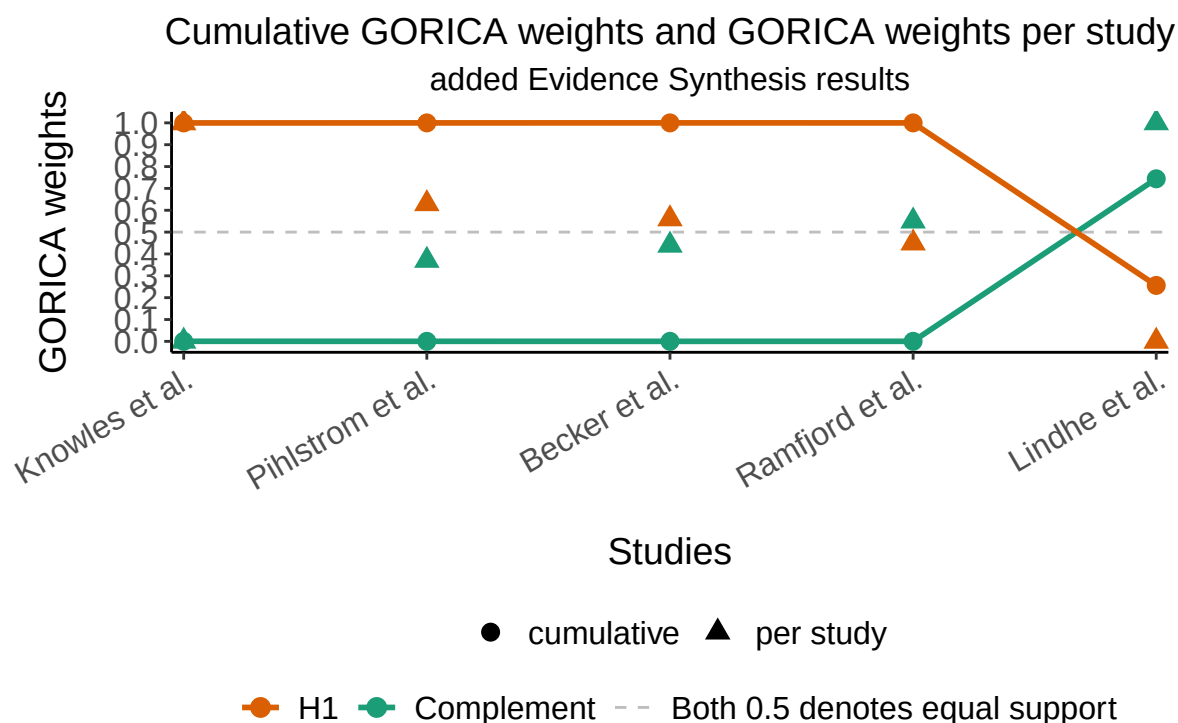
Final GORICA weights:

	H1	Complement
	0.256	0.744

Ratio final GORICA weights:

	vs. H1	vs. Complement
H1	1.000	0.344
Complement	2.907	1.000

```
plot(results_Set3_asc)
```



When you would perform GORICA evidence synthesis for the set of studies leaving the second one out, then this results in:

```
dat_subset <- dat[dat$trial != 2, ]
study_names_subset <- study_names[-1]
results_Set3_asc_subset <- evSyn(object = dat_subset, outcome_col = "outcome", hypotheses = list(H1 = H1, Complement = Complement),
  order_studies = "ascending", study_names = study_names_subset)
results_Set3_asc_subset
```

restriktor (0.6-50): added Evidence Synthesis results:

Input type 'escalc' detected: Parameter estimates and covariance matrix inherited from escalc object

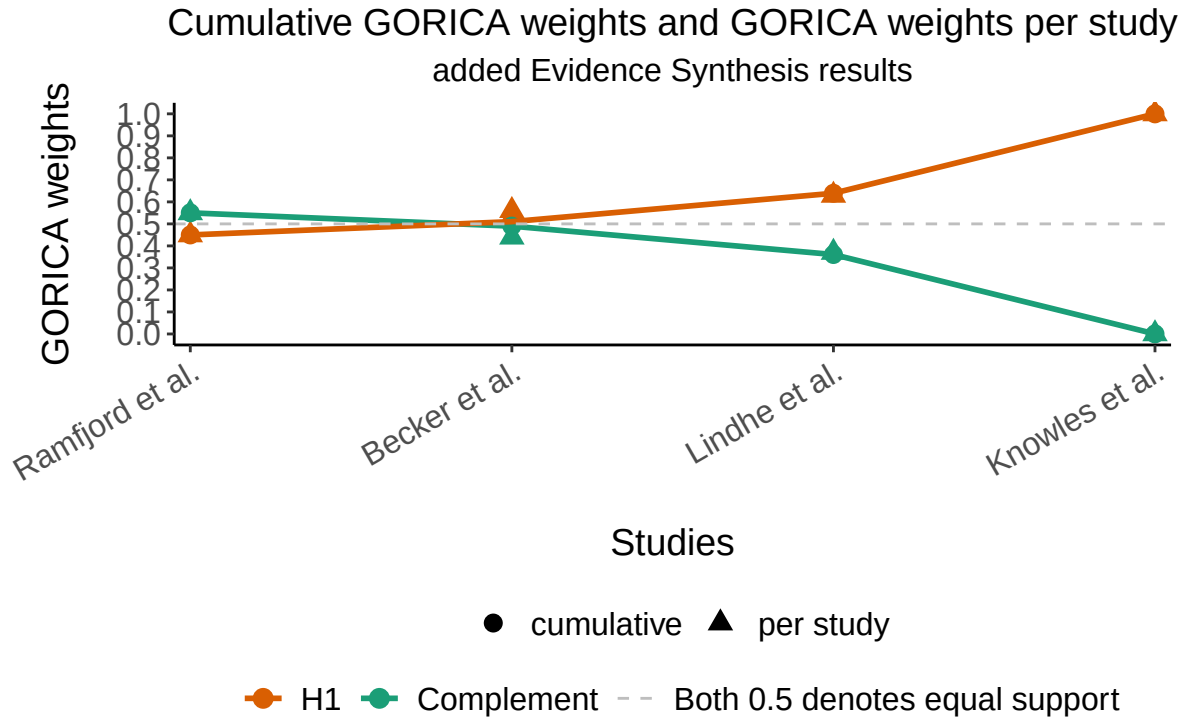
Final GORICA weights:

	H1	Complement
	1.000	1.89e-04

Ratio final GORICA weights:

	vs. H1	vs. Complement
H1	1.000	5.28e+03
Complement	1.89e-04	1.000

```
plot(results_Set3_asc_subset)
```



Specifying sets of hypotheses in `evSyn`

In the examples above, the set of hypotheses (reflecting the central theories) are the same for all studies. This does not need to be the case. Note: The number of hypotheses per study-specific set is always the same, but the specification of the (study-specific) hypotheses does not need to be the same for all studies.

If the specification of hypotheses differ per study, then one needs to specify the sets of hypotheses per study. Below, one can find examples of code to specify sets of hypotheses in the `evSyn` function in the case of having (for ease) 3 primary studies; for both the cases where there are two or 1 central theories (excluding the failsafe).

Below, you can find the following three blocks of examples:

- for having one set of 2 hypotheses for all 3 studies;
- for having 3 study-specific sets (containing 2 study-specific hypotheses);
- including some incorrect specification;
- for having 3 study-specific sets of 1 hypothesis.

```
# If you have one set of hypotheses for all studies,
# then you should specify one list.
# You can use one of the following:
#
Hypo_studies <- list(H1, H2)
# names central theories: H1 and H2
#
```

```

Hypo_studies <- list(Hpos, Hneg)
# names central theories: H1 and H2
#
Hypo_studies <- list(Hpos = H1, Hneg = H2)
# names central theories: Hpos and Hneg
#
# Note that, in each study,
# the parameter estimate names should be the same.

# If you have study-specific sets for your 3 studies,
# then you should specify 3 lists in one list.
#
# Note that the order of hypotheses in the set matters,
# as well as the labeling of the central theories.
# The labeling of the hypotheses sets does not matter.
#
# You can use one of the following:
#
Hypo_studies <- list(list(H1_St1, H2_St1),
                    list(H1_St2, H2_St2),
                    list(H1_St3, H2_St3))
# names central theories: H1 and H2
#
Hypo_studies <- list(SetStudy1 = list(H1_St1, H2_St1),
                    SetStudy2 = list(H1_St2, H2_St2),
                    SetStudy3 = list(H1_St3, H2_St3))
# names central theories: H1 and H2
# Note: the names 'SetStudy.' will not be used.
#
Hypo_studies <- list(list(Hpos = H1_St1, Hneg = H2_St1),
                    list(Hpos = H1_St2, Hneg = H2_St2),
                    list(Hpos = H1_St3, Hneg = H2_St3))
# names central theories: Hpos and Hneg
#
Hypo_studies <- list(SetStudy1 = list(Hpos = H1_St1, Hneg = H2_St1),
                    SetStudy2 = list(Hpos = H1_St2, Hneg = H2_St2),
                    SetStudy3 = list(Hpos = H1_St3, Hneg = H2_St3))
# names central theories: Hpos and Hneg
# Note: the names 'SetStudy.' will not be used.

# INcorrect ways of specifying:
Hypo_studies <- list(list(Hpos = H1_St1, Hneg = H2_St1),
                    list(H1 = H1_St2, H2 = H2_St2),
                    list(Ha = H1_St3, Hb = H2_St3))
# Note: Each central theory should have the same labeling.
#
Hypo_studies <- list(list(Hpos = H1_St1, Hneg = H2_St1),
                    list(Hpos = H1_St2, Hneg = H2_St2),
                    list(Hneg = H1_St3, Hpos = H2_St3))
# Note: The order of hypotheses in the sets should be the same.

# Just to be sure,
# if there would be one central theory for all 3 studies:

```

```
#
Hypo_studies <- list(list(H1_St1),
                     list(H1_St2),
                     list(H1_St3))
# names central theories: H1
#
Hypo_studies <- list(list(Hpos = H1_St1),
                     list(Hpos = H1_St2),
                     list(Hpos = H1_St3))
# names central theories: Hpos
```

Types of input for evSyn

Estimates and their variance-covariance matrices are not necessary to perform evidence synthesis. The `evSyn` function also works with GORIC(A) values or GORIC(A) weights, as well as with log-likelihood and penalty values. This is shown, using Set 1 of Example 2, in the next subsections.

Notably, when using GORIC(A) values, GORIC(A) weights, or log-likelihood and penalty values as input, you do not need to specify the hypotheses in `evSyn`. That is because the hypotheses were provided previously when calculating the values/weights with `goric`. Nevertheless, to avoid confusion, `hypo_names` can be used for a better labeling in the output, as will be demonstrated below.

using Estimates and Covariance Matrix

As shown above, one can use estimates and their variance-covariance matrices as input for GORIC(A) evidence synthesis:

```
results_Set1 <- evSyn(object = estimates, VCOV = covmats,
                     hypotheses = list(H1.1 = H1.1, H1.2 = H1.2)
                     #comparison = "unconstrained" # default
                     #type_ev = "added" # default
                     )
```

```
results_Set1
```

```
restriktor (0.6-50): added Evidence Synthesis results:
```

```
Input type 'est_vcov' detected: Parameter estimates and covariance matrix
```

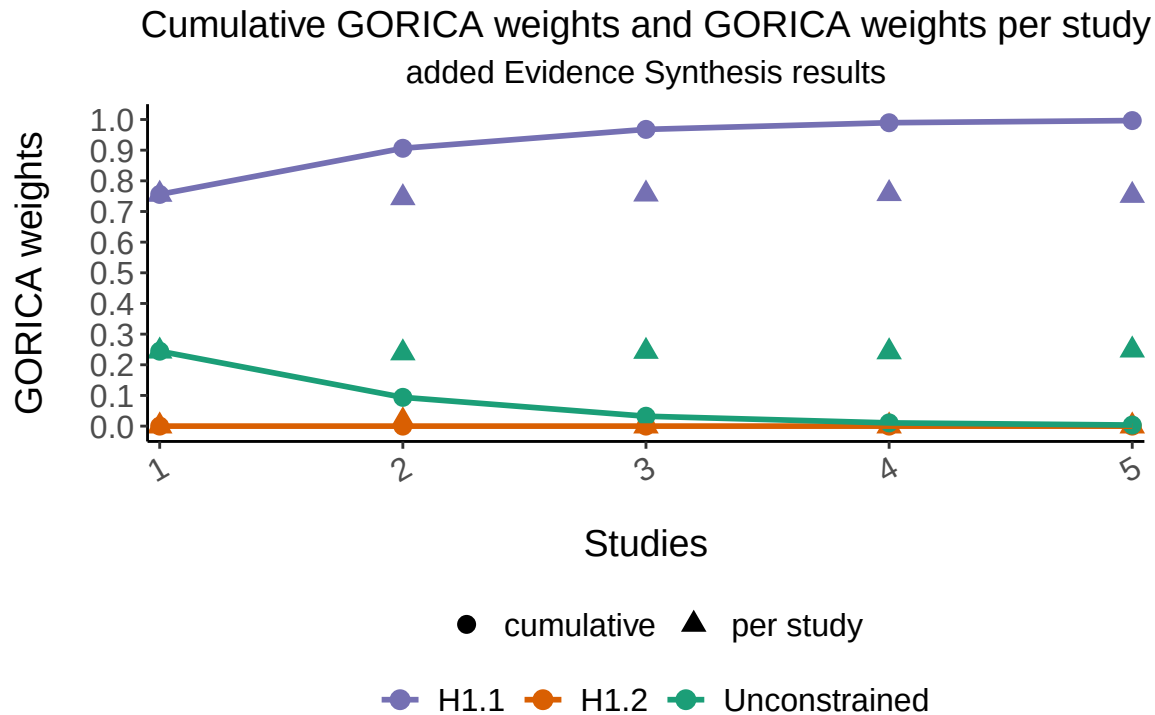
```
Final GORICA weights:
```

	H1.1	H1.2	Unconstrained
---	0.997	0.000	0.003

```
Ratio final GORICA weights:
```

	vs. H1.1	vs. H1.2	vs. Unconstrained
H1.1	1.000	4.95e+34	285.368
H1.2	0.000	1.000	0.000
Unconstrained	0.004	1.74e+32	1.000

```
# summary(results_Set1)
plot(results_Set1)
```



using GORIC(A) Values

The GORIC(A) values can be used as input by creating a vector with the `c()` command; e.g., `c(gorica_H1, gorica_H2, gorica_Hu)`.

Below, the GORICA values are subtracted from a `goric` object.

```
# goric objects needed to obtain the GORICA values from
gorica_Study1 <- goric(estimates[[1]], VCOV = covmats[[1]], hypotheses = list(H1.1 = H1.1,
  H1.2 = H1.2))
gorica_Study2 <- goric(estimates[[2]], VCOV = covmats[[2]], hypotheses = list(H1.1 = H1.1,
  H1.2 = H1.2))
gorica_Study3 <- goric(estimates[[3]], VCOV = covmats[[3]], hypotheses = list(H1.1 = H1.1,
  H1.2 = H1.2))
gorica_Study4 <- goric(estimates[[4]], VCOV = covmats[[4]], hypotheses = list(H1.1 = H1.1,
  H1.2 = H1.2))
gorica_Study5 <- goric(estimates[[5]], VCOV = covmats[[5]], hypotheses = list(H1.1 = H1.1,
  H1.2 = H1.2))

# list of GORICA values for each of the five studies
GORICA <- list(gorica_Study1$result[, 4], gorica_Study2$result[, 4], gorica_Study3$result[,
  4], gorica_Study4$result[, 4], gorica_Study5$result[, 4])
# GORICA

# GORICA evidence synthesis
results_Set1_gorica <- evSyn(GORICA)
#
results_Set1_gorica
```

restriktor (0.6-50): added Evidence Synthesis results:

Input type 'icvalues' detected: Information criteria values

Final GORICA weights:

	H1	H2	H3
	0.997	0.000	0.003

Ratio final GORICA weights:

	vs. H1	vs. H2	vs. H3
H1	1.000	4.95e+34	285.368
H2	0.000	1.000	0.000
H3	0.004	1.74e+32	1.000

```
# summary(results_Set1_gorica) plot(results_Set1_gorica)
# round(results_Set1_gorica$Final_ratio_GORICA_weights, 2)
```

The results lead us to the same output as above of course.

In cases where hypotheses names may be confusing, as it is here, it is recommendable that the user utilize the `hypo_names` function argument to make the results more accessible. In this example, the function would be:

```
results_Set1_gorica <- evSyn(GORICA, hypo_names = c("H1.1", "H1.2", "Hu"))
#
results_Set1_gorica
```

restriktor (0.6-50): added Evidence Synthesis results:

Input type 'icvalues' detected: Information criteria values

Final GORICA weights:

	H1.1	H1.2	Hu
	0.997	0.000	0.003

Ratio final GORICA weights:

	vs. H1.1	vs. H1.2	vs. Hu
H1.1	1.000	4.95e+34	285.368
H1.2	0.000	1.000	0.000
Hu	0.004	1.74e+32	1.000

```
# summary(results_Set1_gorica) plot(results_Set1_gorica)
# round(results_Set1_gorica$Final_ratio_GORICA_weights, 2)
```

using GORIC(A) Weights

Similarly, the GORIC(A) weights can be used:

```
# list of GORICA weights for each of the five studies
GORICAweights <- list(gorica_Study1$result[, 7], gorica_Study2$result[, 7], gorica_Study3$result[,
  7], gorica_Study4$result[, 7], gorica_Study5$result[, 7])
# GORICAweights

# GORICA evidence synthesis
results_Set1_gw <- evSyn(GORICAweights, hypo_names = c("H1.1", "H1.2", "Hu"))
#
results_Set1_gw
```

```
restriktor (0.6-50): added Evidence Synthesis results:
```

```
Input type 'icweights' detected: Information criteria weights (summing to 1)
```

```
Final GORICA weights:
```

	H1.1	H1.2	Hu
	0.997	0.000	0.003

```
---
```

```
Ratio final GORICA weights:
```

	vs. H1.1	vs. H1.2	vs. Hu
H1.1	1.000	4.95e+34	285.368
H1.2	0.000	1.000	0.000
Hu	0.004	1.74e+32	1.000

```
# summary(results_Set1_gw) plot(results_Set1_gw)
# round(results_Set1_gw$Final_ratio_GORICA_weights, 2)
```

Note

Instead of using GORIC(A) weights, one could also use posterior model probabilities (rendered from Bayesian model selection).

using Log-likelihood and Penalty Values

One can also perform evidence aggregation using log-likelihood and penalty values.

Here, these values (as vectors) are retrieved from the `goric` functions ran before. It is also possible to concatenate numerical values using `c()` and create two vectors: one containing the log-likelihood values of each study and one containing the penalties of each study. Then, you should make two lists of the study-specific vectors, which then should be fed into `evSyn`, in the `object` and `PT` arguments, respectively.

```
# Log-likelihood values
LL_Set1 <- list(gorica_Study1$result[,2], gorica_Study2$result[,2],
               gorica_Study3$result[,2], gorica_Study4$result[,2],
               gorica_Study5$result[,2])
# Penalty values
PT_Set1 <- list(gorica_Study1$result[,3], gorica_Study2$result[,3],
               gorica_Study3$result[,3], gorica_Study4$result[,3],
               gorica_Study5$result[,3])

# GORICA evidence synthesis
results_Set1_LLandPT <- evSyn(LL_Set1, PT = PT_Set1,
                             #type_ev = "added", # Default
                             hypo_names = c("H1.1", "H1.2", "Hu"))
#
results_Set1_LLandPT
```

```
restriktor (0.6-50): added Evidence Synthesis results:
```

```
Input type 'll_pt' detected: Log-likelihood and penalty values
```

```
Final GORICA weights:
```

	H1.1	H1.2	Hu
--	------	------	----

```
0.997 0.000 0.003
---
```

Ratio final GORICA weights:

	vs. H1.1	vs. H1.2	vs. Hu
H1.1	1.000	4.95e+34	285.368
H1.2	0.000	1.000	0.000
Hu	0.004	1.74e+32	1.000

```
# summary(results_Set1_LLandPT)
#plot(results_Set1_LLandPT)
#round(results_Set1_LLandPT$Final_ratio_GORICA_weights, 2)
```

Once more, one can see that the results do not change when the function arguments changed (as it denotes the same input). Any of these methods is suitable for aggregating evidence over multiple studies.

As in the previous sections, it is not necessary to specify the hypotheses in `evSyn`. However, to avoid confusion, `hypo_names` is used for a better labeling in the output.